

# SQL Queries

Final queries for E3, M4 and H4

---

Round 1, Phase 2: Analytical Core

**Competition:** Data Vortex | AARUUSH'26  
**Theme:** Rebuilding the Social Engine  
**Team:** Entropy  
**Members:** Saanvi Grover & Aditya Sharma  
  
**Questions:** E3 · M4 · H4  
**Database:** SQLite (Entropy\_social\_engine.db)

## Database Schema

All three queries run against the two-table SQLite schema built in the notebook from the Phase 1 cleaned CSVs. Constraints are enforced at the DDL level (not just in pandas), so every query can rely on them.

```
1 PRAGMA foreign_keys = ON;
2
3 CREATE TABLE users (
4     user_id      TEXT PRIMARY KEY,
5     location     TEXT NOT NULL,
6     language     TEXT NOT NULL,
7     account_created DATE NOT NULL,
8     follower_count INTEGER NOT NULL CHECK (follower_count >= 0)
9 );
10
11 CREATE TABLE posts (
12     post_id      TEXT PRIMARY KEY,
13     user_id      TEXT NOT NULL REFERENCES users(user_id),
14     platform     TEXT, -- NULL where corrupted (14.9%)
15     text_content TEXT, -- NULL where corrupted (14.3%)
16     timestamp    DATETIME NOT NULL,
17     likes        INTEGER CHECK (likes >= 0), -- NULL where corrupted (15.1%)
18     shares       INTEGER NOT NULL CHECK (shares >= 0),
19     comments     INTEGER NOT NULL CHECK (comments >= 0)
20 );
21
22 CREATE INDEX idx_posts_user_id ON posts(user_id);
23 CREATE INDEX idx_posts_platform ON posts(platform);
24 CREATE INDEX idx_posts_timestamp ON posts(timestamp);
25 CREATE INDEX idx_users_language ON users(language);
26 CREATE INDEX idx_users_location ON users(location);
```

**Row counts:** users = 1,500; posts = 12,000. Three columns keep the NULLs introduced by the corruption because Phase 1 showed they are Missing Completely at Random (Mann–Whitney U,  $p = 0.64$ ); each query below states explicitly how it treats them.

## E3 – Average Engagement by Platform (Easy)

**Challenge.** Calculate the average likes, shares, and comments for each platform. Which platform generates the highest average total engagement?

```
1 -- E3: Average Engagement by Platform
2 -- Which platform generates the highest average total engagement?
3 -- Posts with a missing platform or missing likes are excluded so that all
4 -- four averages are computed over the same set of rows.
5
6 WITH platform_stats AS (
7     SELECT
8         platform,
9         COUNT(*) AS post_count,
10        ROUND(AVG(likes), 1) AS avg_likes,
11        ROUND(AVG(shares), 1) AS avg_shares,
12        ROUND(AVG(comments), 1) AS avg_comments,
13        ROUND(AVG(likes + shares + comments), 1) AS avg_total_engagement
14 FROM posts
15 WHERE platform IS NOT NULL
16        AND likes IS NOT NULL
17 GROUP BY platform
18 ),
19 overall AS (
20     SELECT AVG(likes + shares + comments) AS grand_avg
```

```

21     FROM posts
22     WHERE platform IS NOT NULL AND likes IS NOT NULL
23 )
24 SELECT
25     RANK() OVER (ORDER BY ps.avg_total_engagement DESC) AS rank,
26     ps.platform,
27     ps.post_count,
28     ps.avg_likes,
29     ps.avg_shares,
30     ps.avg_comments,
31     ps.avg_total_engagement,
32     ROUND(100.0 * (ps.avg_total_engagement - o.grand_avg) / o.grand_avg, 2)
33     AS pct_vs_overall
34 FROM platform_stats ps
35 CROSS JOIN overall o
36 ORDER BY rank;

```

**Output columns.** rank, platform, post\_count, avg\_likes, avg\_shares, avg\_comments, avg\_total\_engagement, pct\_vs\_overall. Returns 5 rows; Instagram ranks first at 4,040.0.

## M4 – Platform Behaviour by High-Follower Users (Medium)

**Challenge.** Among users with at least 30,000 followers, determine which platform gives them the highest average engagement per post.

```

1  -- M4: Platform Behaviour by High-Follower Users
2  -- Among users with at least 30,000 followers, which platform gives them the
3  -- highest average engagement per post? Each platform's figure is compared
4  -- with the same platform's average across ALL users to see whether the
5  -- high-follower cohort behaves any differently.
6
7  WITH high_follower_posts AS (
8      SELECT
9          p.platform,
10         p.user_id,
11         p.likes + p.shares + p.comments AS engagement
12     FROM posts p
13     JOIN users u ON u.user_id = p.user_id
14     WHERE u.follower_count >= 30000
15         AND p.platform IS NOT NULL
16         AND p.likes IS NOT NULL
17 ),
18 cohort_by_platform AS (
19     SELECT
20         platform,
21         COUNT(DISTINCT user_id) AS high_follower_users,
22         COUNT(*) AS posts,
23         ROUND(AVG(engagement), 1) AS avg_engagement_high_followers
24     FROM high_follower_posts
25     GROUP BY platform
26 ),
27 baseline_by_platform AS (
28     SELECT
29         platform,
30         ROUND(AVG(likes + shares + comments), 1) AS avg_engagement_all_users
31     FROM posts
32     WHERE platform IS NOT NULL AND likes IS NOT NULL
33     GROUP BY platform
34 )
35 SELECT
36     RANK() OVER (ORDER BY c.avg_engagement_high_followers DESC) AS rank,
37     c.platform,

```

```

38     c.high_follower_users,
39     c.posts,
40     c.avg_engagement_high_followers,
41     b.avg_engagement_all_users,
42     ROUND(c.avg_engagement_high_followers - b.avg_engagement_all_users, 1)
43                                     AS cohort_minus_baseline
44 FROM cohort_by_platform c
45 JOIN baseline_by_platform b ON b.platform = c.platform
46 ORDER BY rank;

```

**Output columns.** rank, platform, high\_follower\_users, posts, avg\_engagement\_high\_followers, avg\_engagement\_all\_users, cohort\_minus\_baseline. Returns 5 rows; Instagram ranks first at 4,145.2 for the 594-user cohort.

## H4 – Follower-to-Engagement Anomaly (Hard)

**Challenge.** Find users who have fewer than 5,000 followers but whose total post engagement places them among the top 10% of all users.

```

1  -- H4: Follower-to-Engagement Anomaly
2  -- Users with fewer than 5,000 followers whose total post engagement places
3  -- them in the top 10% of all users.
4  --
5  -- Level 1: aggregate every post to its author (total engagement per user).
6  -- Level 2: rank every user against the whole population (decile + percentile).
7  -- Level 3: keep the top decile, then apply the low-follower filter.
8  --
9  -- Missing likes are treated as 0 (COALESCE) so that a corrupted like count
10 -- never removes a whole post's shares and comments from a user's total.
11
12 WITH user_engagement AS (
13     SELECT
14         u.user_id,
15         u.location,
16         u.follower_count,
17         COUNT(p.post_id) AS post_count,
18         SUM(COALESCE(p.likes, 0) + p.shares + p.comments) AS total_engagement
19     FROM users u
20     JOIN posts p ON p.user_id = u.user_id
21     GROUP BY u.user_id, u.location, u.follower_count
22 ),
23 ranked_users AS (
24     SELECT
25         *,
26         NTILE(10) OVER (ORDER BY total_engagement DESC) AS engagement_decile,
27         PERCENT_RANK() OVER (ORDER BY total_engagement DESC) AS pct_rank,
28         COUNT(*) OVER () AS total_users
29     FROM user_engagement
30 )
31 SELECT
32     RANK() OVER (ORDER BY total_engagement DESC) AS rank_among_anomalies,
33     user_id,
34     location,
35     follower_count,
36     post_count,
37     total_engagement,
38     ROUND(total_engagement * 1.0 / post_count, 1) AS avg_engagement_per_post,
39     ROUND(100.0 * pct_rank, 2) AS top_percentile
40 FROM ranked_users
41 WHERE engagement_decile = 1 -- top 10% of all users by total engagement
42     AND follower_count < 5000 -- ...despite a small audience
43 ORDER BY total_engagement DESC;

```

**Output columns.** `rank_among_anomalies`, `user_id`, `location`, `follower_count`, `post_count`, `total_engagement`, `avg_engagement_per_post`, `top_percentile`. Returns 16 users; the top-10% threshold is 43,948 total engagement.

## Reproducibility

---

- Engine: SQLite 3 (window functions require  $\geq 3.25$ ), executed through Python's `sqlite3` + `pandas` in `Entropy_01_SQL_Analysis.ipynb`.
- Each query is stored verbatim in `queries/` and the JPEG screenshots in `images/` were rendered from the exact result sets returned by these files (`Entropy_run_queries.py`).
- No query mutates data; all are single read-only `SELECT` statements.